VegaDūta: install and first agent

Step by step on a real machine: free local model, no API key. Screenshots are from the install described here (2026-10-17).

1 Before you start

You need three things on the machine:

- **Docker** with Compose v2, and about 8 GB of memory given to Docker.
- **Ollama** (`brew install ollama` or ollama.com/download) with a model pulled, for example `ollama pull qwen3.5:4b` . This is what makes the install free: no API key.
- **The code:** `git clone https://github.com/cdandlc/vegaduta.git && cd vegaduta`

> Until the first release is published the images are built from your checkout (add `--build` , 20 to 40 minutes the first time). This walkthrough used locally built images, so the build step is skipped.

2 Run the installer

One command. On a first interactive install it asks how agents should get their model. Pressing Enter twice accepts the recommendation (a local model sized to your RAM, plus free web search).

bash ./scripts/vegaduta up --lite

```
./scripts/vegaduta up --lite --build

Which model should power your agents?
  1) Open-weights model on this machine: free, private, no API key  (recommended: qwen3.5:9b via host O
llama)
  2) A cloud provider (OpenAI, Anthropic, ...): add your key in the console after sign-up
  3) Pick a different local model
Choice [1]:
```

You can answer up front instead, which is what a script or this walkthrough does:

bash

```
./scripts/vegaduta up --lite --local-ai=host --local-model qwen3.5:4b --search
```

> `--local-ai=host` uses the Ollama already on your Mac (fast, uses the Apple GPU). Drop `=host` to run Ollama in a container instead. `--search` adds a private SearXNG container so agents get `web_search` with no key. `--dry-run` prints what any combination would do without starting anything.

3 The installer checks your machine, then starts the stack

installer: prerequisites

```
==> Checking prerequisites (tier: lite)
  [ok]   Docker daemon reachable (29.8.2)
  [ok]   docker compose 5.5.1
  [ok]   Docker memory 15 GB (>= 6 GB for lite)
  [ok]   Ollama reachable on this machine (127.0.0.1:11434)
  [ok]   model 'qwen3.5:4b' is pulled
  [ok]   Free disk 411 GB
  [ok]   .env present (.env)
  [ok]   All secrets set
  [ok]   APP_ENCRYPTION_KEY decodes to 32 bytes
  [ok]   .env is owner-only (600)
  [ok]   Release e2e-local
  [ok]   Local mode (no BASE_DOMAIN): URLs on localhost
  [ok]   Ports free: 3000 8080 8180 5432 6379
Looks good (0 warning(s)).
```

It generates every secret itself and writes them to `deploy/compose/.env` (owner-only). **Back that file up:** `APP_ENCRYPTION_KEY` must never change, or stored provider keys and channel secrets can no longer be decrypted.

4 It finishes by testing that the model can run agents

installer: done

```
==> Checking that 'qwen3.5:4b' can run agents (tool calls)...
  [ok]   'qwen3.5:4b' makes structured tool calls: agents with tools will work

VegaDūta is up (lite)

  Console          http://localhost:3000
  API              http://localhost:8080
  Identity (Keycloak) http://localhost:8180
  Local model      qwen3.5:4b via host Ollama (free platform pool)

First login
  1. Open http://localhost:3000 and choose "Sign up" — the first account creates your
     organisation and becomes its tenant admin.
  2. Make that account a platform operator (cross-tenant admin console):
       scripts/vegaduta make-admin you@example.com
  3. Create your first agent. With --local-ai the free local model is already
     registered as the platform pool; otherwise add a provider key
     (Settings -> Providers). No key and no GPU? Open /edge to run a model
     in the browser instead (docs/local-models.md).
  Keycloak admin console: http://localhost:8180/admin

  status: scripts/vegaduta status    logs: scripts/vegaduta logs core    stop: scripts/vegaduta down
```

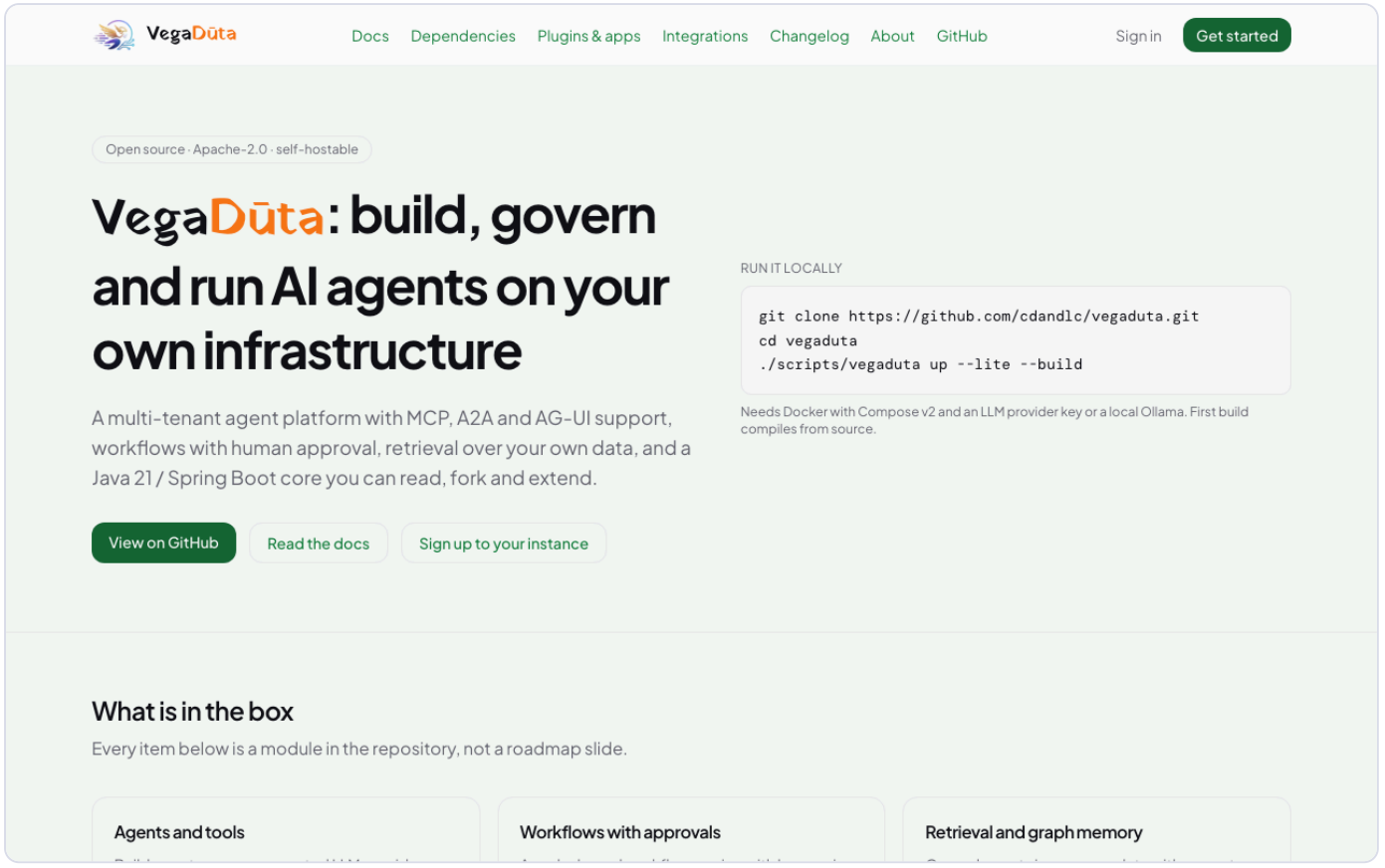
The line *makes structured tool calls* is a real measurement of the model you picked. A model that cannot call tools would be flagged here, before you build an agent on it.

Stack running:

```
bash ./scripts/vegaduta status

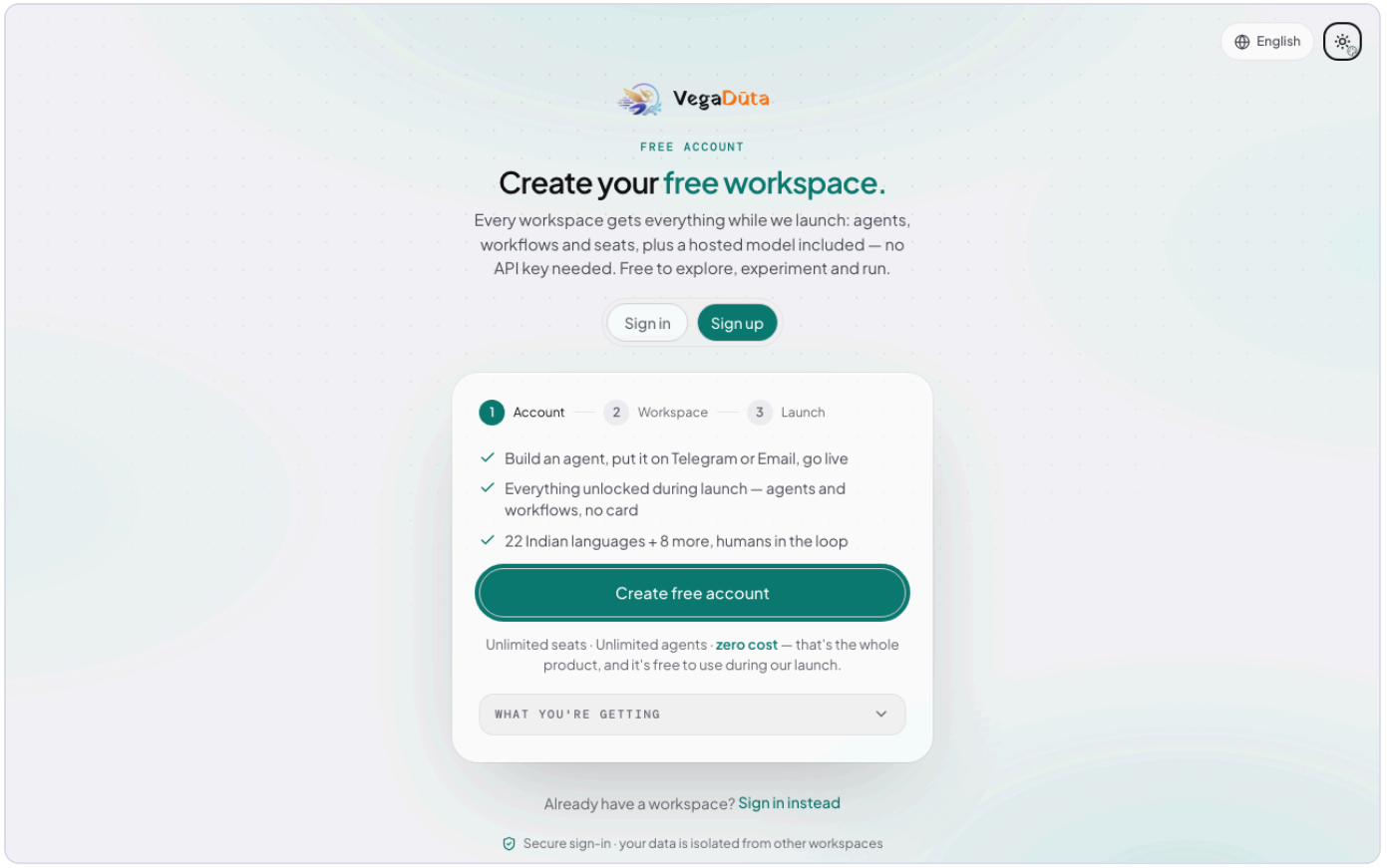
==> Tier: lite
SERVICE    STATE     <no value>   PORTS
core       running   healthy      127.0.0.1:8080->8080/tcp
keycloak   running   healthy      127.0.0.1:8180->8080/tcp
postgres   running   healthy      127.0.0.1:5432->5432/tcp
searxng    running   healthy      8080/tcp
valkey     running   healthy      127.0.0.1:6379->6379/tcp
web        running   healthy      127.0.0.1:3000->3000/tcp
```

5 Open the console

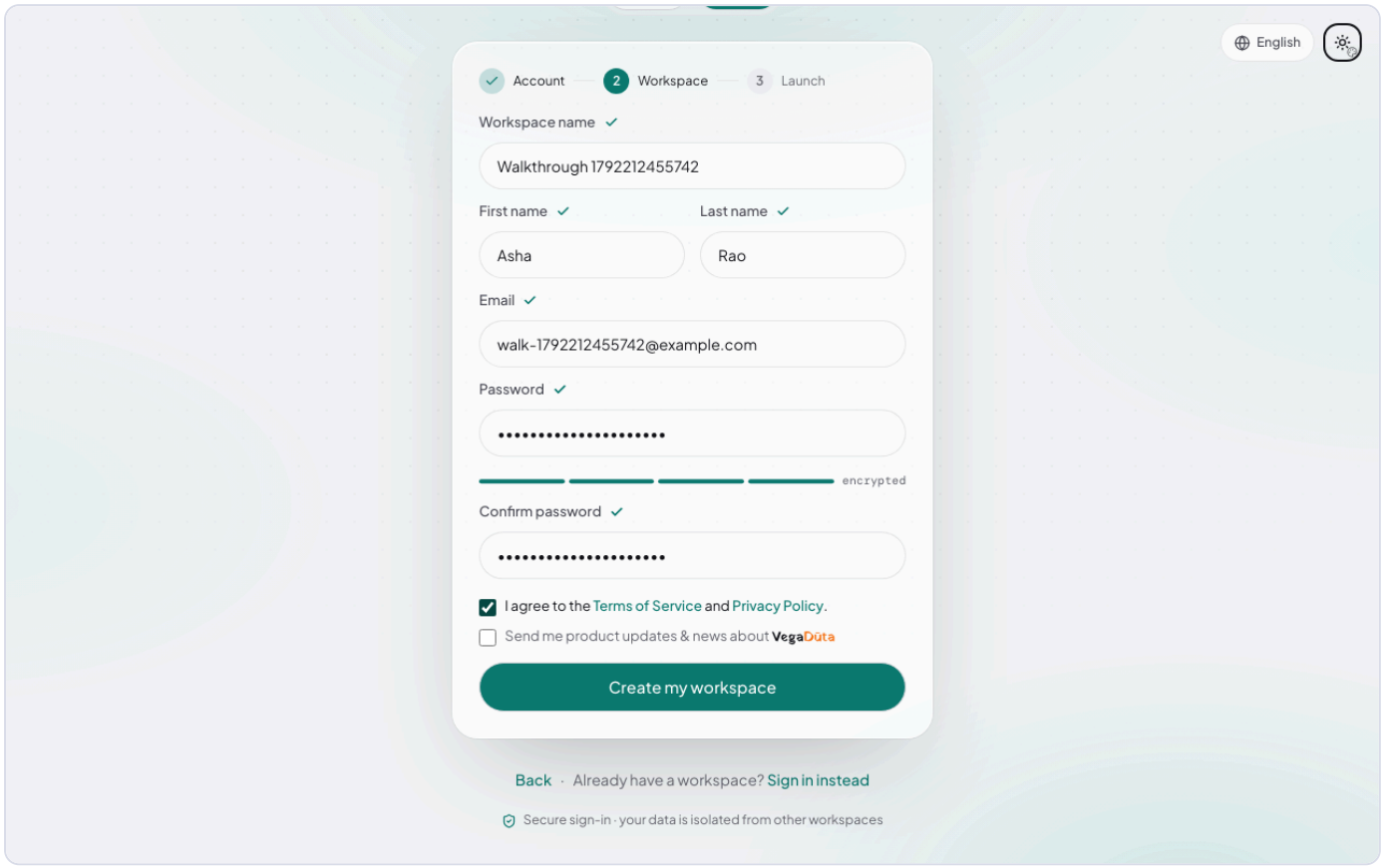


http://localhost:3000, the public page of your own install.

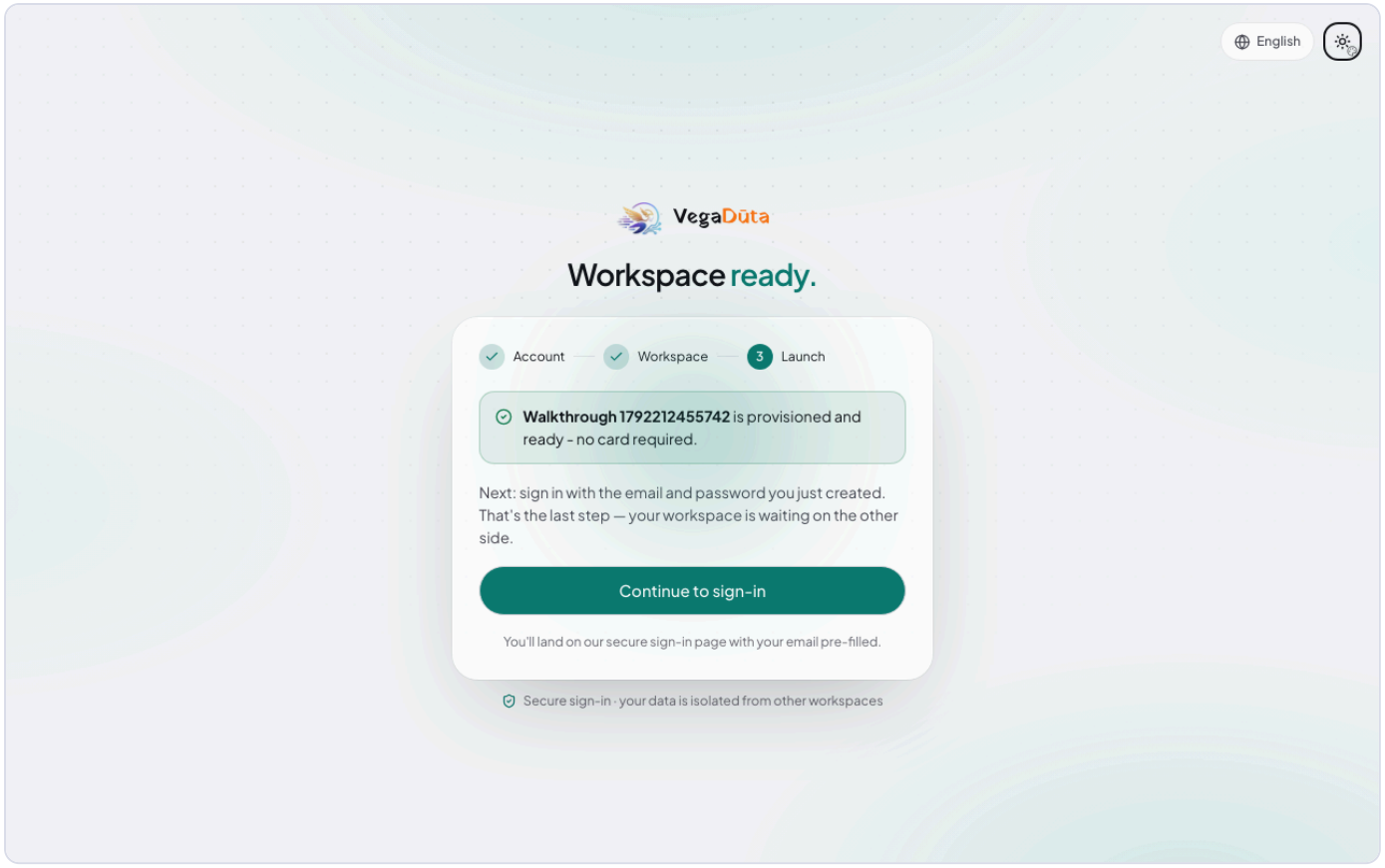
6 Create your workspace



Choose Sign up, then Create free account.

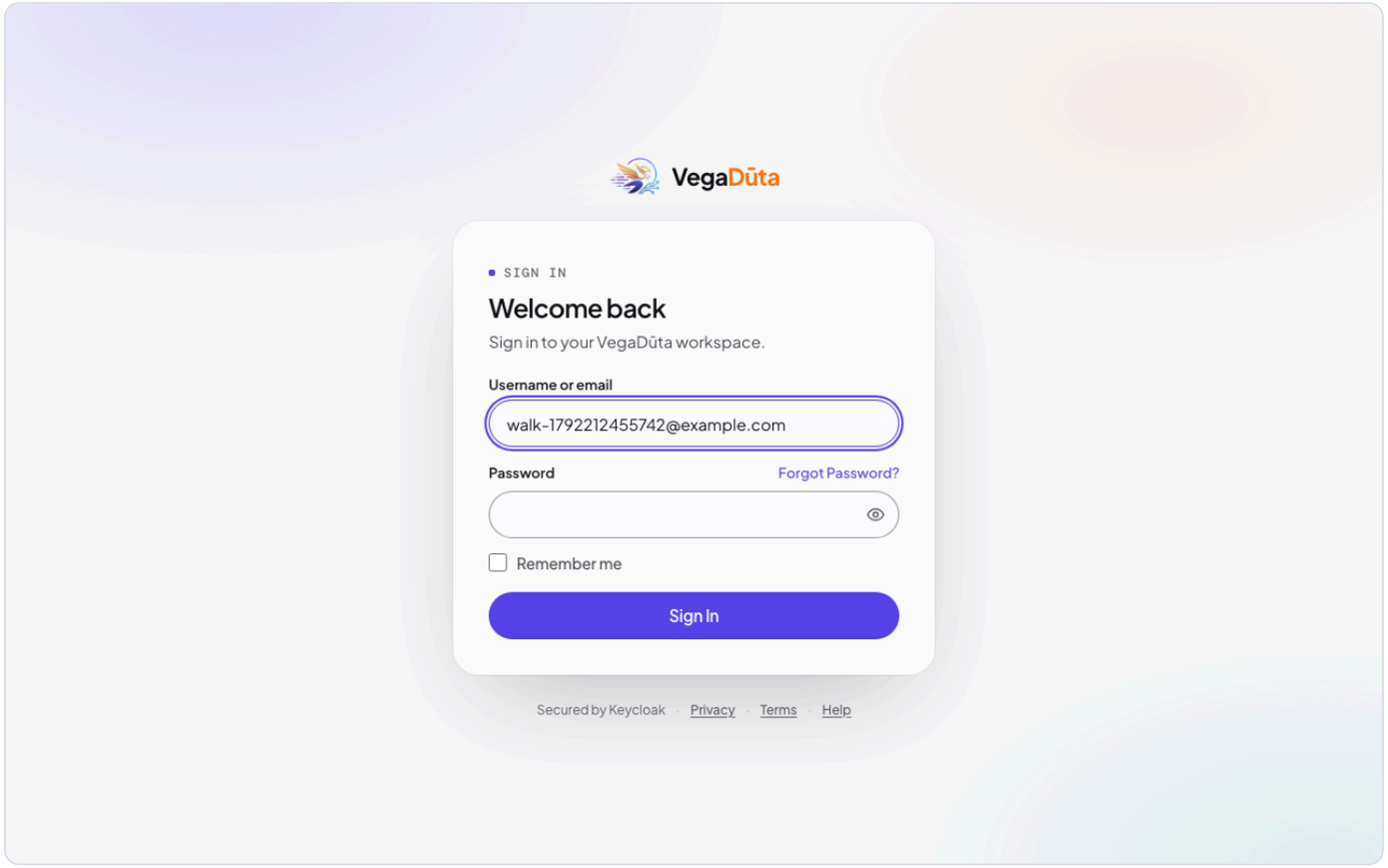


Workspace name, your name, email and password. The first account becomes the workspace admin.

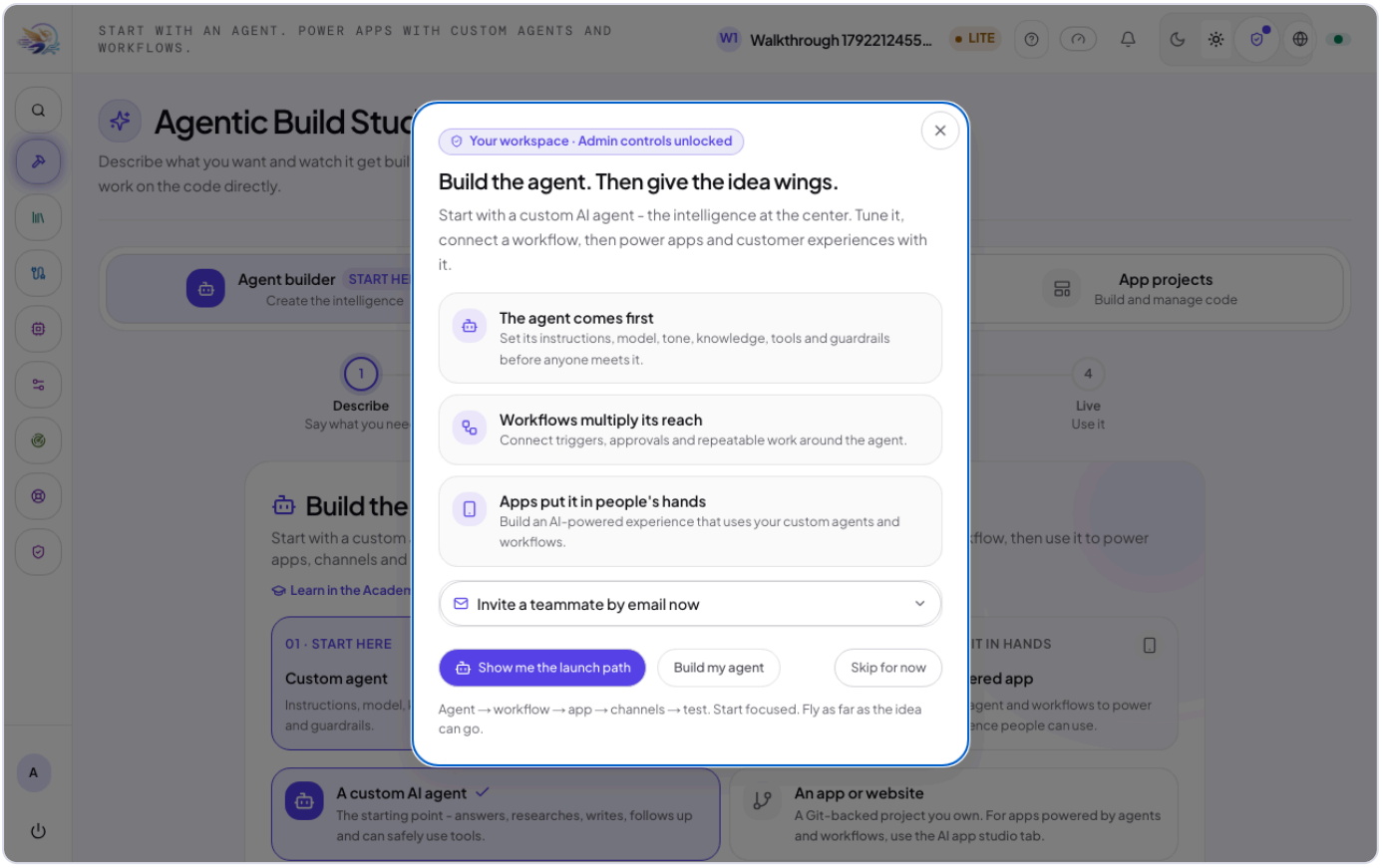


The workspace is provisioned. No card, no email verification on the lite tier.

7 Sign in

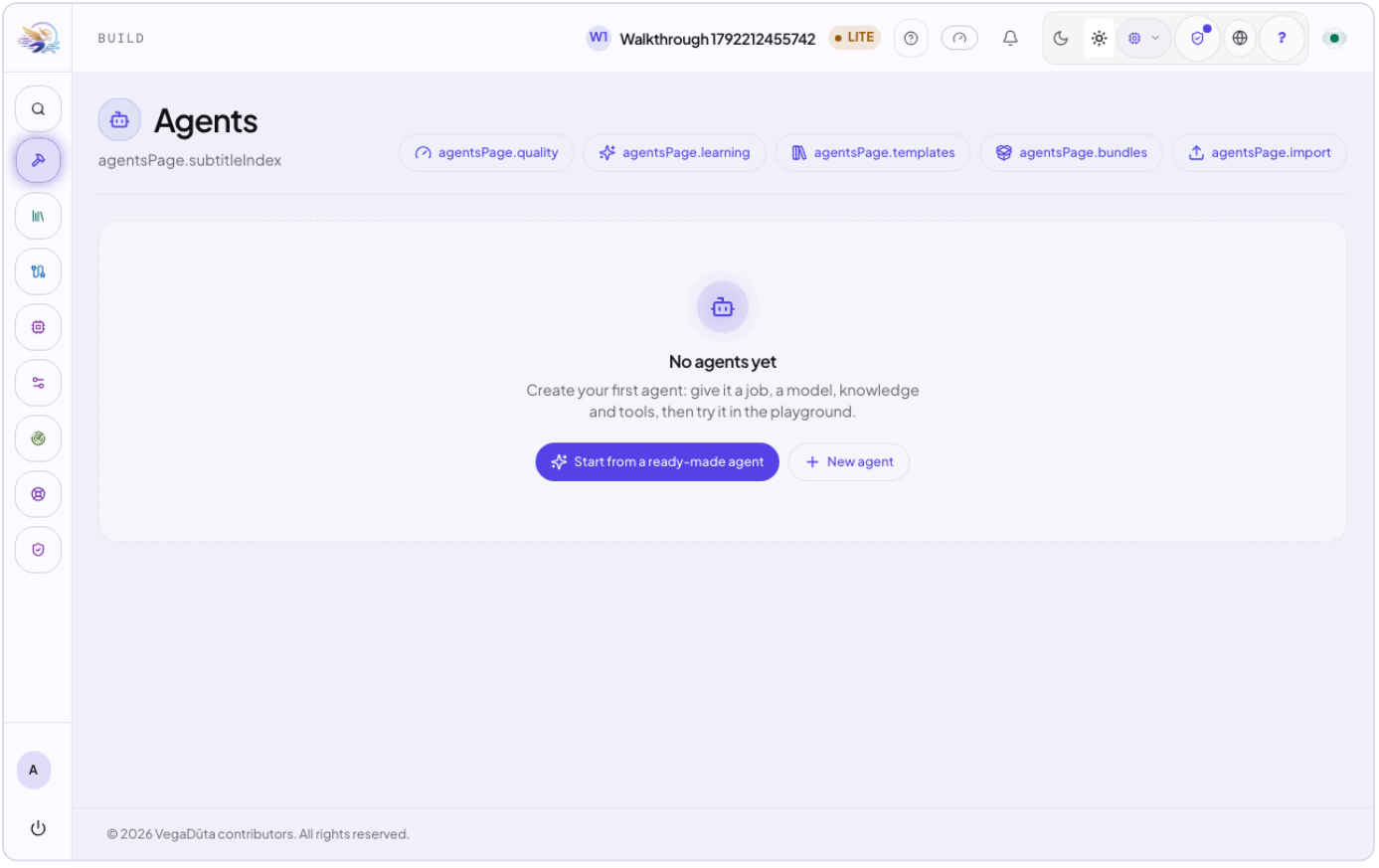


Sign-in is handled by Keycloak, running in the same stack. Your email is pre-filled.

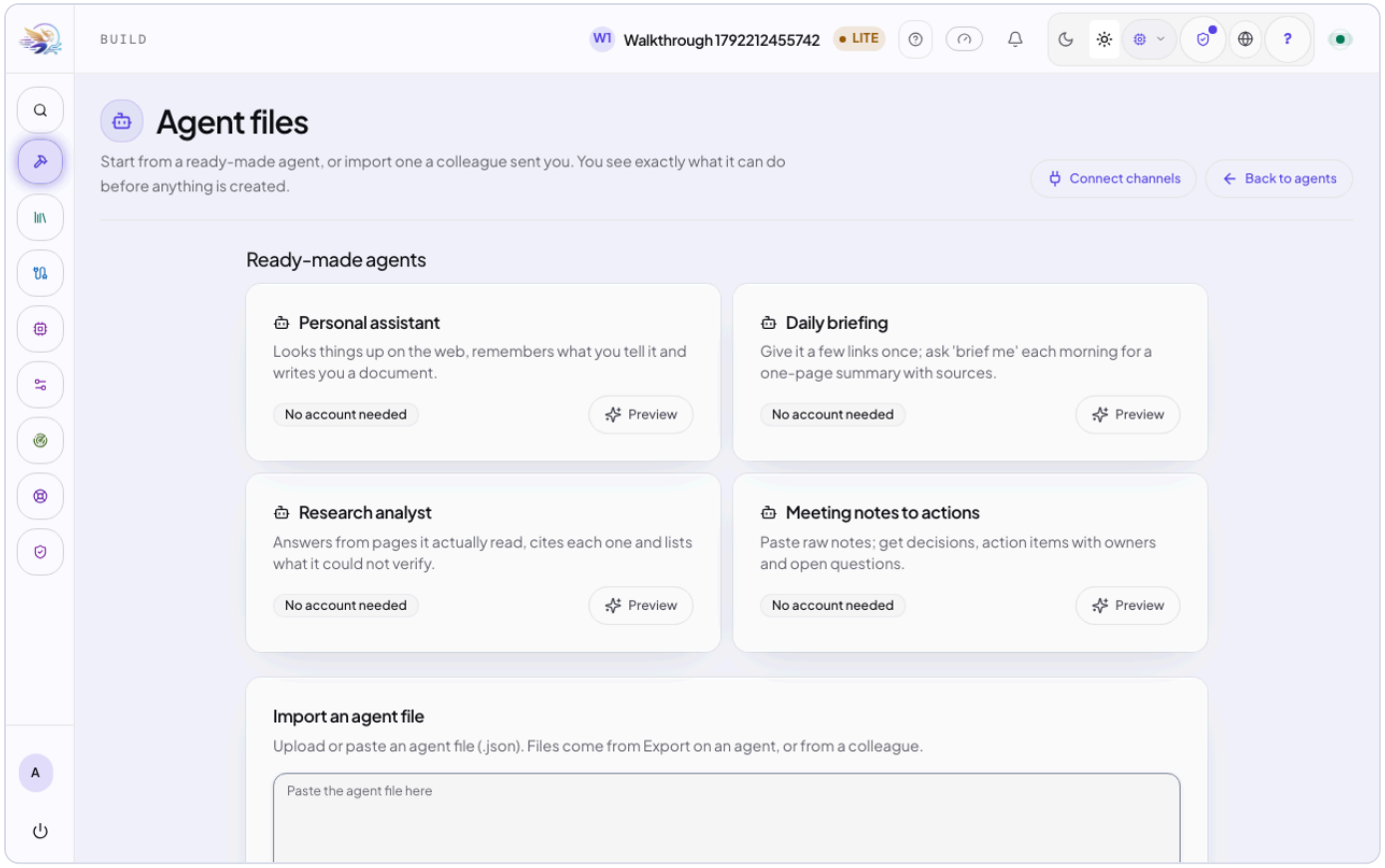


You land in the console.

8 Start from a ready-made agent

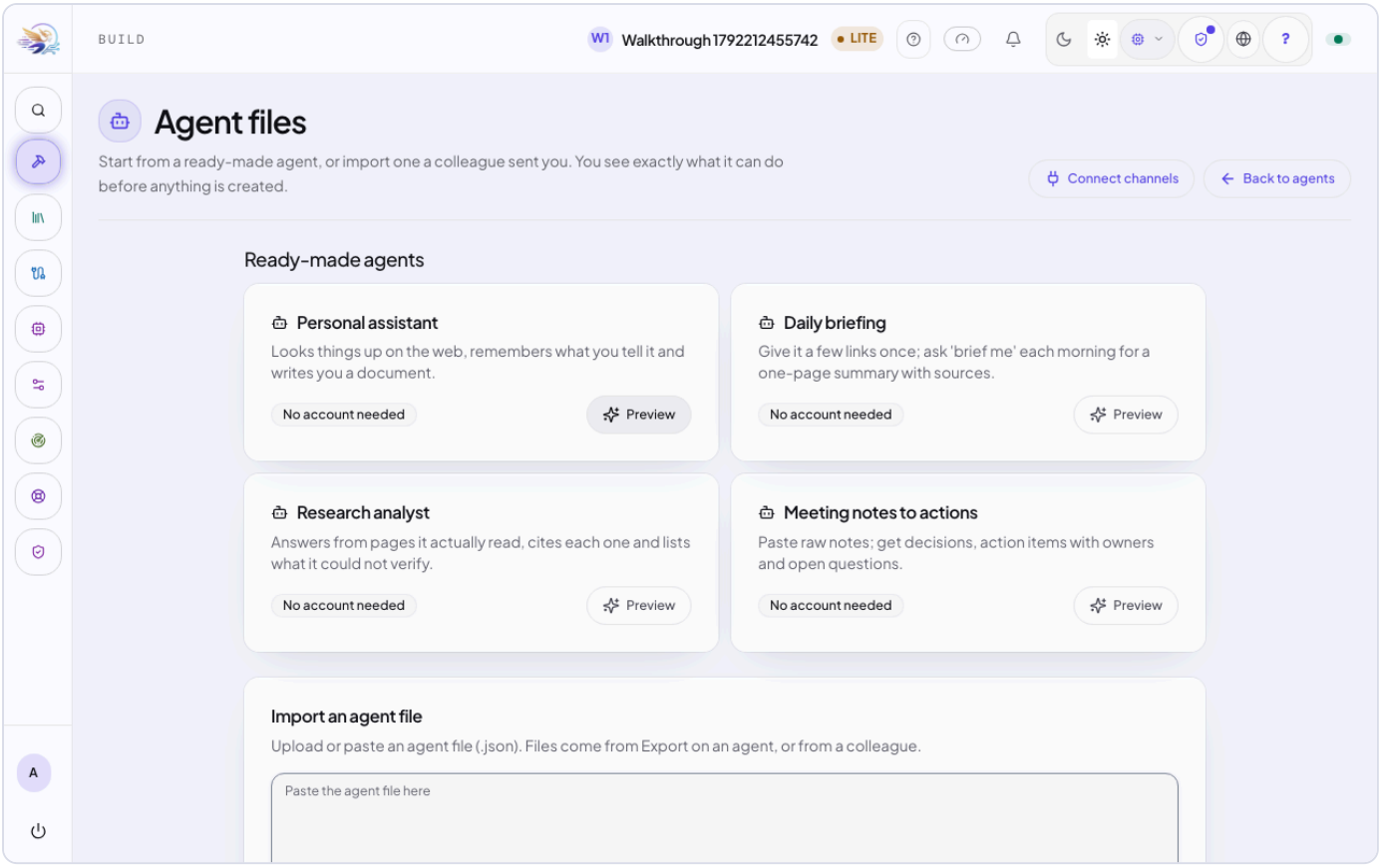


A new workspace has no agents. The empty list offers Start from a ready-made agent, and Agent files in the header.



Agent files: four ready-made agents, or import a file a colleague sent you.

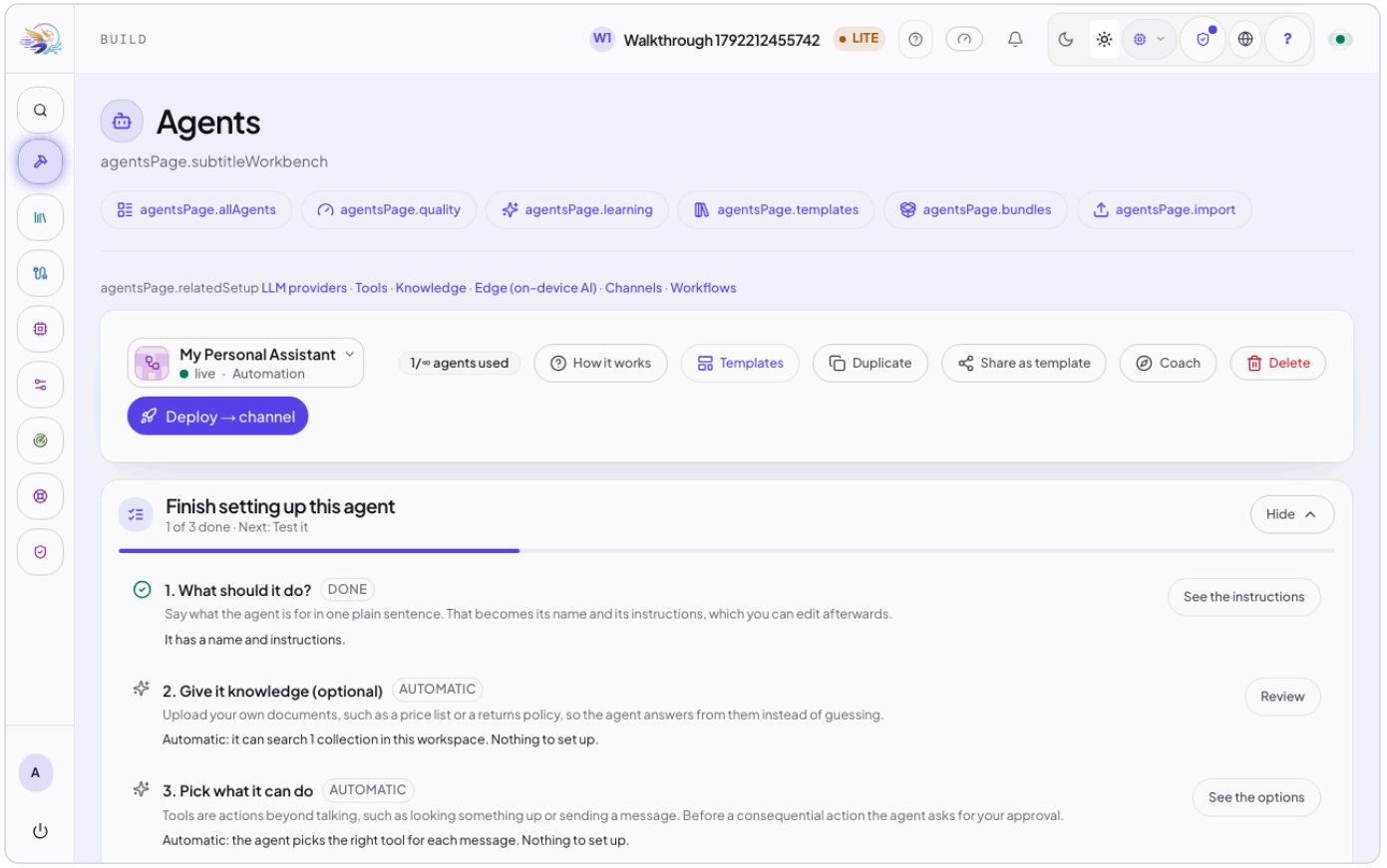
9 Preview before you import



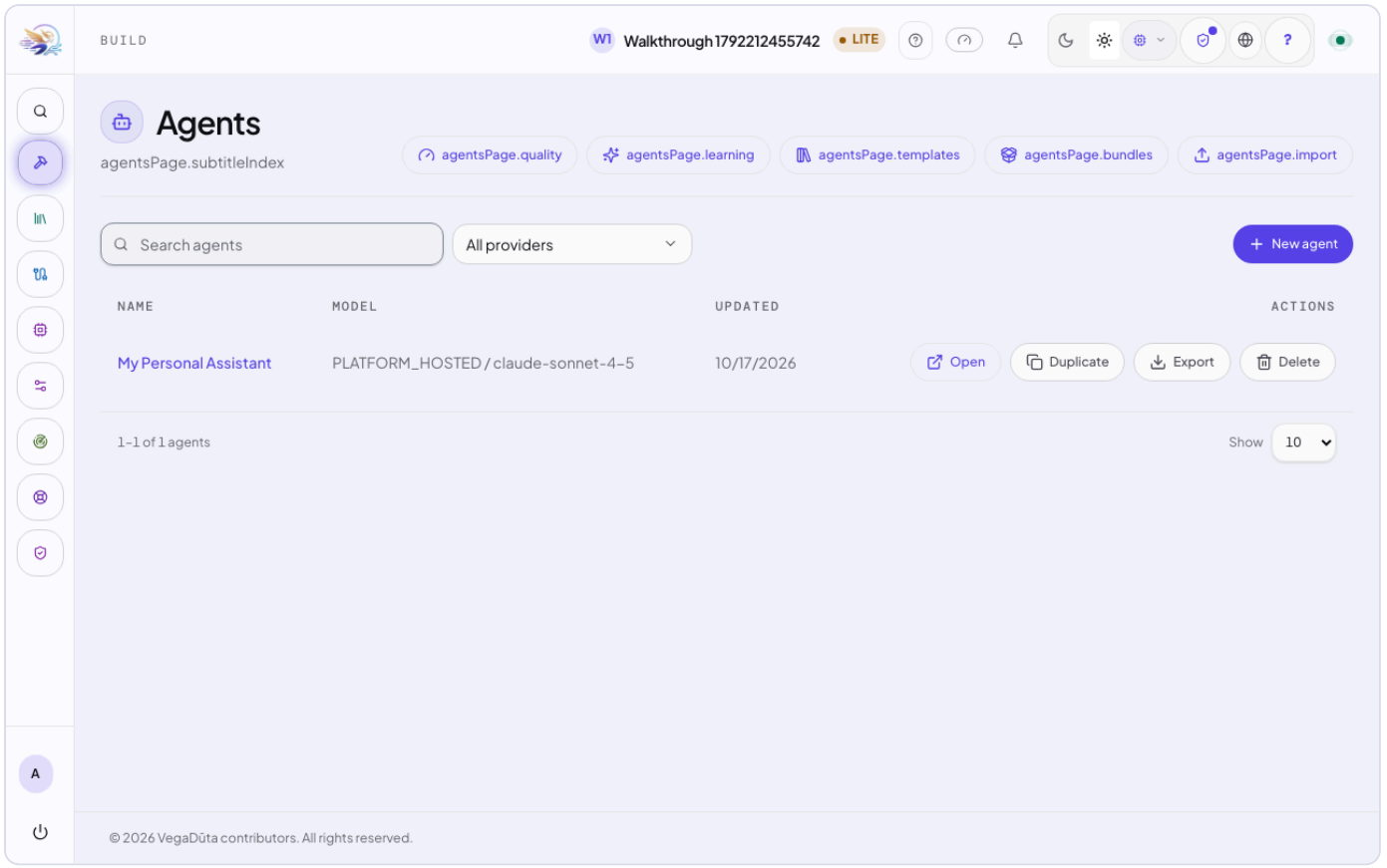
Nothing exists yet. You see the tools the agent can use and the autonomy it will really get (SUPERVISED: anything with a real-world effect waits for your approval). Rename it if you like, then press Import agent.

> On this run the import took a while because creating an agent makes a model call on the local model. Wait for the page to move on.

10 Your agent exists

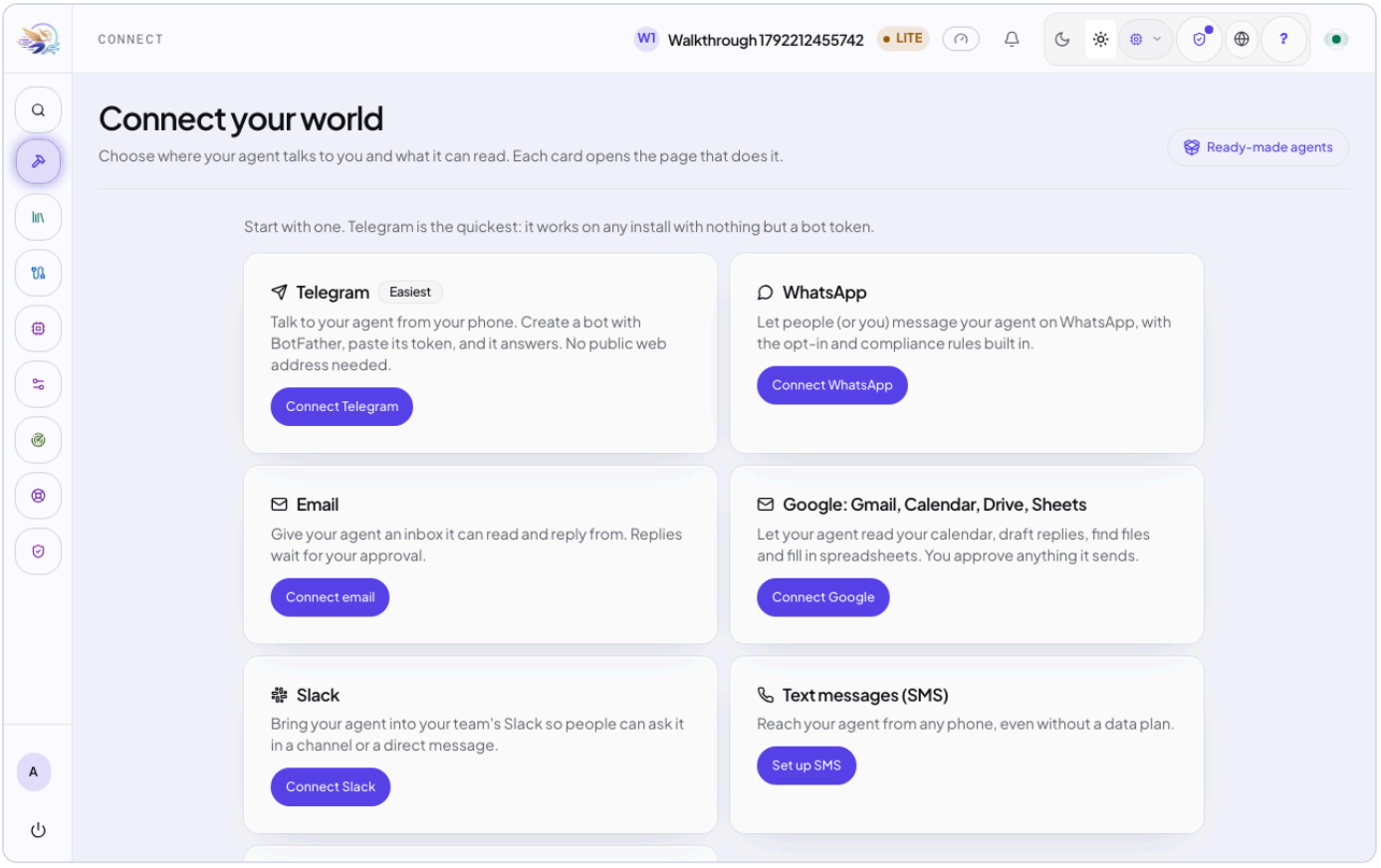


The agent opens in the workbench, running on the platform pool, which on this install is your local model.

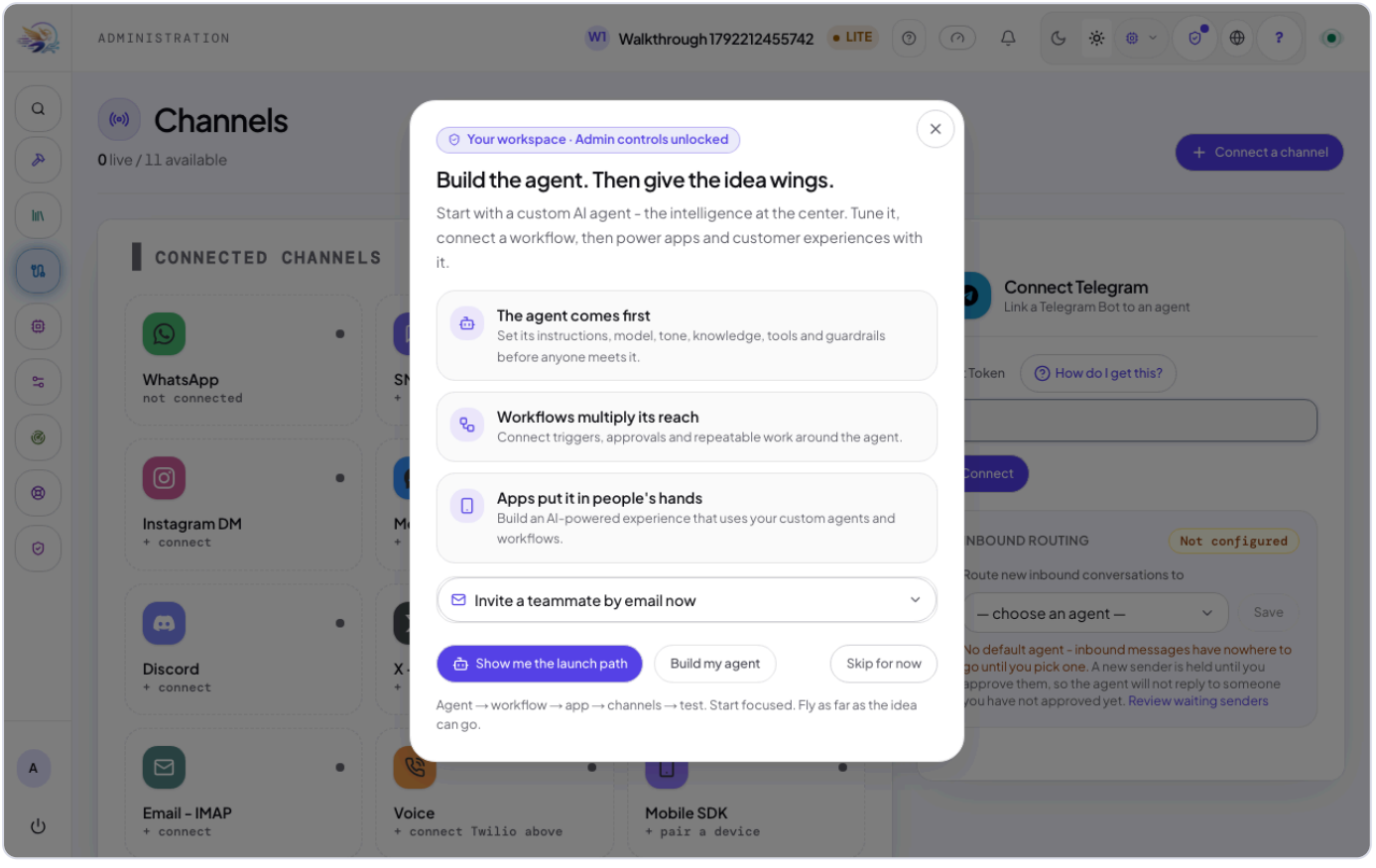


The list now has Export next to Duplicate and Delete. Export downloads the agent as a shareable <name>.agent.json file with no secrets in it.

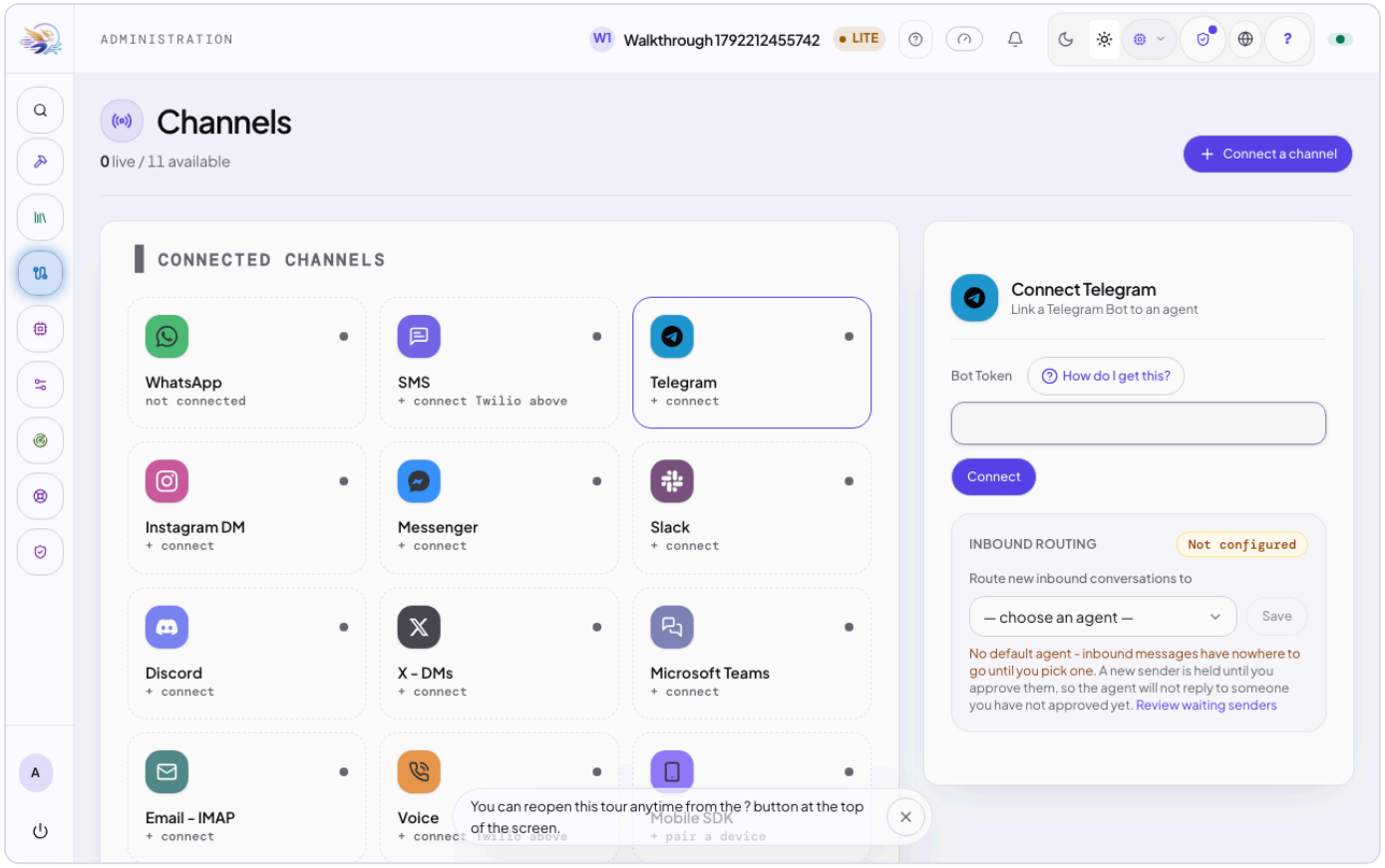
11 Connect your world



Connect lists Telegram, WhatsApp, email, Google, Slack, SMS and your documents. Each card shows whether it is connected and opens the page that connects it.

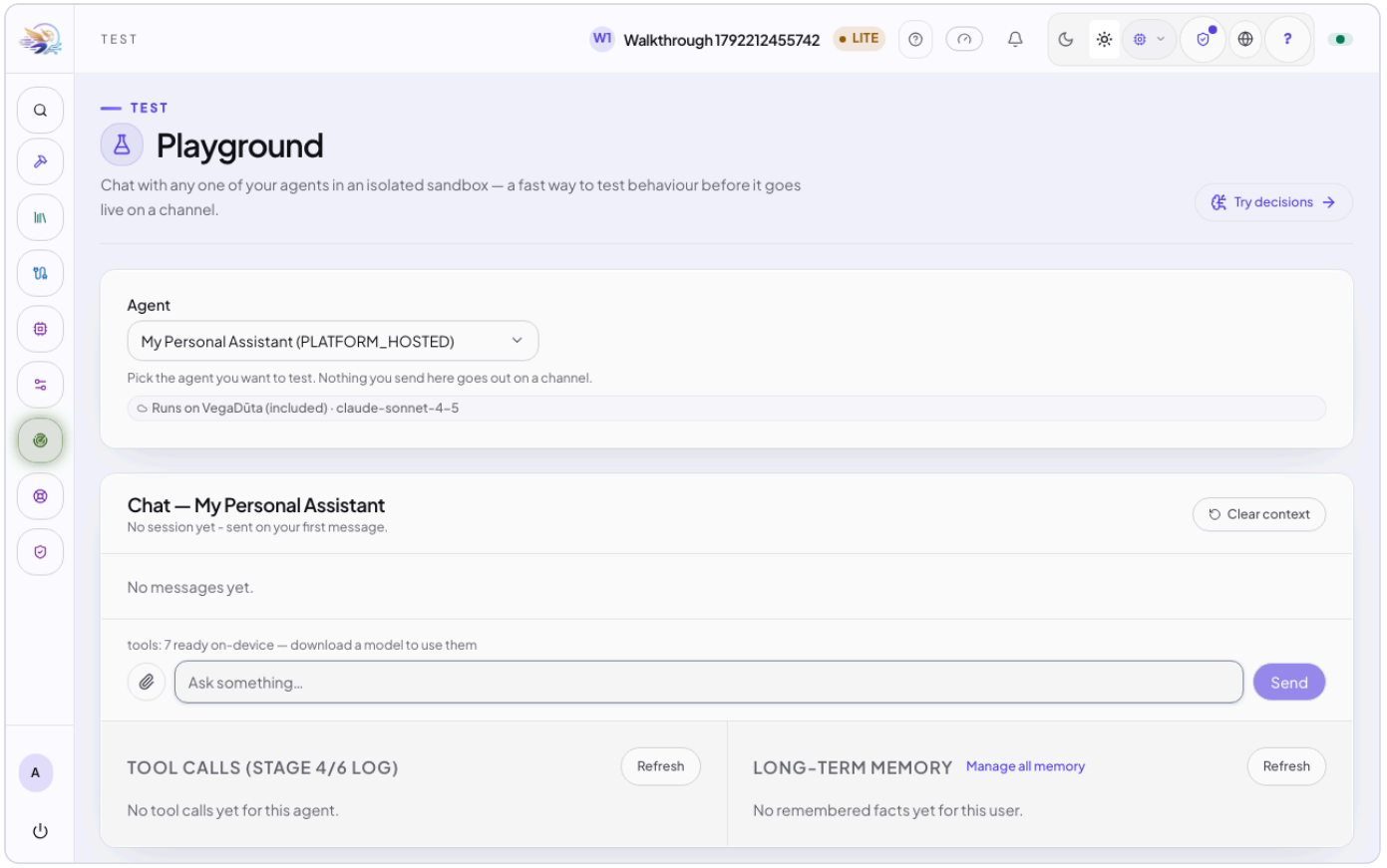


The first visit to Channels shows a welcome dialog. Skip for now.

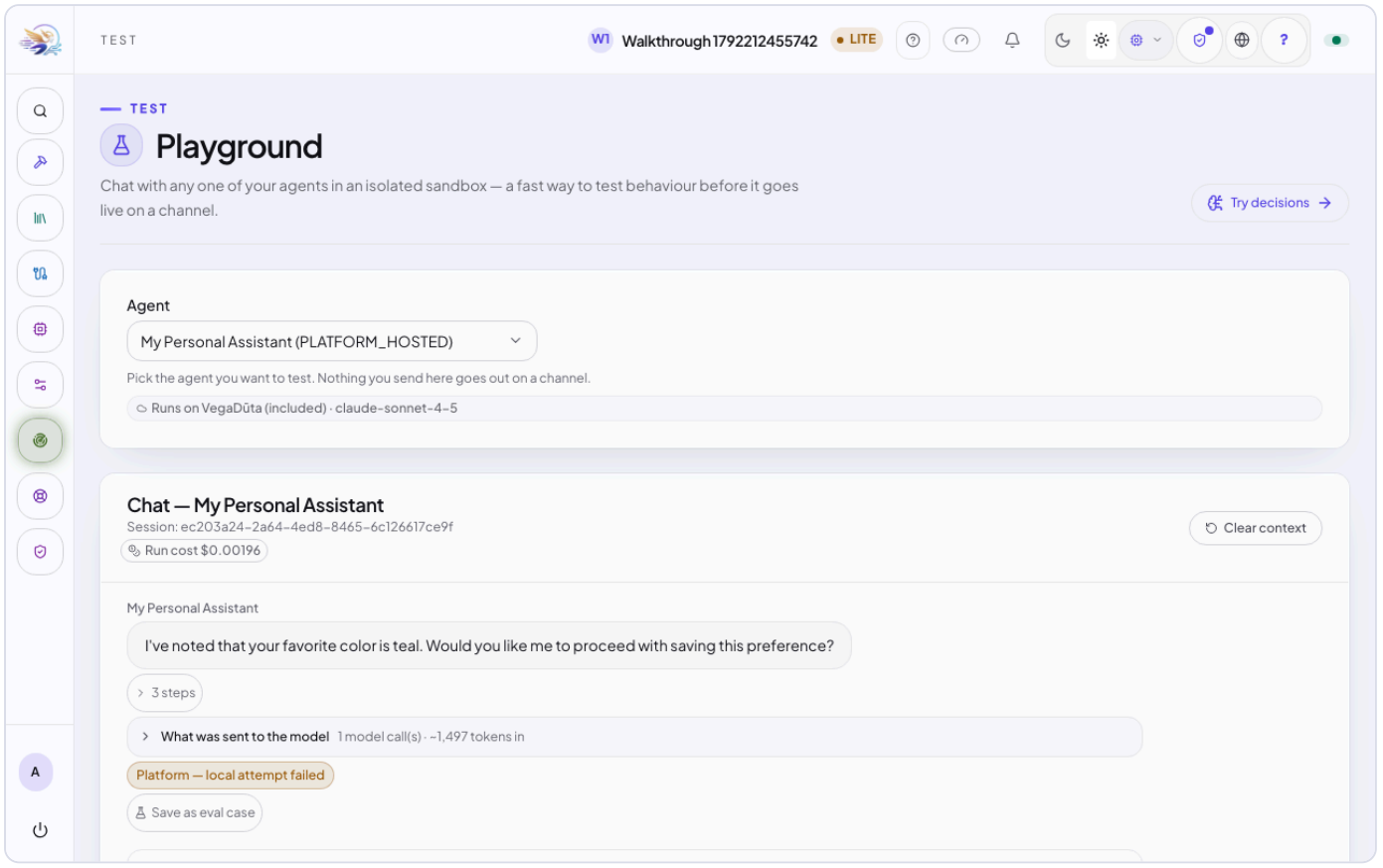


Telegram is the quickest: create a bot with @BotFather, paste the token here, and on a lite install it answers with no public web address (core polls Telegram itself).

12 Talk to your agent



Playground: pick the agent and send a message.



The local model answered, and the agent decided to call its remember_fact tool. Because the agent is SUPERVISED, the action is paused until you approve it (the Approve and run button just below). That pause is the safety design working.

13 What to do next

- **Connect a channel** (Connect page) and set a default agent for it so inbound messages have somewhere to go.
- **Add your documents** under Knowledge so the agent answers from your own material.
- **Try a bigger model** if tool use is shaky: `./scripts/vegaduta models list` then `models pull qwen3:8b`.
- **Share an agent:** Export it, send the file; the other person imports it from Agent files.
- **Stop / start:** `./scripts/vegaduta down` and `up`. `down --volumes` deletes your data.

14 Honest notes from this run

This walkthrough was produced by installing and using the stack on a real machine, which found problems. Fixed in the repository:

- **Free web search broke the lite install.** `--search` left the core container off the default network, so it could not reach the database. Fixed, with a regression test.
- **First sign-up ended on "Failed to send email".** Lite has no mail server but required email verification. The lite tier now turns it off (`KEYCLOAK_REQUIRE_EMAIL_VERIFICATION`), with a test.
- **The end-to-end test helper was out of date** for the self-hosted sign-up screen. Updated.

Still open, not fixed:

- **Creating or importing an agent can take over a minute** on a local model (a model call happens during creation).
- **The Playground label is wrong on a local install:** it says "claude-sonnet-4-5" and shows a small run cost, although the local model answered and local calls are free.
- The Playground shows "Platform - local attempt failed" first: the console tries an in-browser model, which fails, then falls back to the platform pool.
- The Telegram bot, WhatsApp, Google and email connections were **not** tested with real accounts; only that their pages open and are reachable from Connect.
- Re-installing with new secrets over an old database volume fails with a confusing "password authentication failed"; use a new project name or remove the old volumes.